



Set de Práctica 2

Objetivos

- Comprender e implementar ordenamiento lineal.
- Comprender el funcionamiento de los árboles binarios de búsqueda e implementar operaciones básicas.
- Resolver problemas utilizando árboles binarios de búsqueda.

Árbol Binario de Búsqueda (ABB o BST)

Un árbol binario de búsqueda (BST por su sigla en inglés) es una estructura de datos que se compone de múltiples nodos, los cuales tienen una **key** que los representa, y una referencia a su padre, a su hijo izquierdo y a su hijo derecho. Al único nodo que no tiene padre, se le conoce como raíz y es desde donde comienza todo el árbol, y a un nodo que no tenga hijos se le conoce como hoja.

La propiedad especial del BST es que el hijo izquierdo es **estrictamente menor** que su padre y el hijo derecho es **estrictamente mayor** que su padre. Esto permite realizar búsquedas, inserciones y eliminaciones de forma eficiente, en particular, $\mathcal{O}(\log n)$ si el árbol se encuentra relativamente balanceado, y $\mathcal{O}(n)$ en otro caso.

Para resolver la mayoría de los ejercicios dentro de este *set*, tendrás que utilizar esta estructura, por lo que te proveemos su implementación base en *LibEDD* con algunas funciones básicas que te podrían ser de ayuda. A continuación, se muestra la definición de los **structs** que componen los BST en la librería (archivo `src/libedd/libedd_bst.h`):

```
libedd_bst.h

1 typedef struct bst_node {
2     int key;
3     struct bst_node *parent;
4     struct bst_node *left;
5     struct bst_node *right;
6 } BstNode;
7
8 typedef char (*MovementFunction)(BstNode *, int);
9
10 typedef struct bst {
11     BstNode *root;
12     size_t size;
13     MovementFunction move_to;
14 } Bst;
```

Como podemos ver, `BstNode` se compone de los elementos de un nodo descritos anteriormente, donde la `key` es de tipo `int`. Por otro lado, vemos que un `Bst` como tal se compone simplemente de una referencia al nodo raíz y una variable `size` que indica la cantidad de nodos dentro del árbol. Para efectos de este *set*, se puede ignorar la `MovementFunction`, que permite definir qué se considera como estrictamente menor y mayor, dado que no se utilizará.

Por otro lado, dispondrás de las siguientes funciones base:

1. `BstNode *bst_node_create(int key)`: Función que crea un nuevo `BstNode` con su atributo `key` igual al argumento del mismo nombre, y retorna un puntero al nuevo nodo.
2. `int bst_node_destroy(EddError *err, BstNode *node)`: Función que elimina el nodo `node`.
3. `Bst *bst_create(MovementFunction move_to)`: Función que crea un nuevo `Bst` con raíz igual a `NULL` y cantidad de nodos igual a 0.
4. `void bst_destroy(EddError *err, Bst *bst)`: Función que elimina por completo el árbol binario de búsqueda `bst`.
5. `void bst_print(EddError *err, Bst *bst, FILE *output_file)`: Función que, dado un `BST bst`, imprime sus contenidos en el archivo `output_file`: raíz, cantidad de nodos, y una representación en texto del árbol completo. En caso de querer imprimir en consola, puede llamarse a la función con `output_file` igual a `NULL`.

Cabe destacar que a veces en los archivos de input se llamará al evento `LOG`, esto gatillará una llamada a `bst_print` para ver que la estructura del árbol sea la esperada.

Ejercicios

DICTSORT {N} {K}

Deberás encargarte de implementar un algoritmo para ordenar de forma alfabética las `N` palabras que se te entregarán posteriormente, **con complejidad asintótica lineal**. Todas las palabras entregadas serán de largo `K`.

Tras ordenar las palabras deberás mostrar el siguiente output:

output	
1	Se han ordenado las {N} palabras con DictSort:
2	{elemento_0}
3	{elemento_1}
4	.
5	.
6	.
7	{elemento_n}

INSERT {X}

Deberás encargarte de insertar el número `X` en el `ABB`.

Tras insertar el número deberas mostrar el siguiente output:

output	
1	Se ha insertado el numero {X} en el ABB.

SEARCH {X}

Deberás encargarte de buscar el número X en el ABB.

1. En caso de encontrar el número deberás mostrar el siguiente output:

output	
1	Se ha encontrado el numero {X} en el ABB.

2. En caso de no encontrar el número deberás mostrar el siguiente output:

output	
1	No se ha encontrado el numero {X} en el ABB.

DELETE {X}

Deberás encargarte de buscar y eliminar el número X en el ABB.

1. En caso de eliminar exitosamente el número deberás mostrar el siguiente output:

output	
1	Se ha eliminado el numero {X} en el ABB.

2. En caso de no poder eliminar el número (no se encontró) deberás mostrar el siguiente output:

output	
1	No se pudo eliminar el numero {X} en el ABB.

TWOSUM {N} {K}

Se te entregarán a continuación N números enteros, con los cuales deberás armar un ABB. Una vez construido el árbol, deberás utilizarlo para determinar si existen dos elementos distintos dentro de él cuya suma sea igual a K.

1. En caso de encontrar un par de números que sumen K, deberás mostrar el siguiente output:

output	
1	Se encontraron dos numeros que suman {K}: {X} y {Y}.

En donde X corresponde al menor entre ambos números, e Y es el mayor.

2. En caso de no encontrar ningún par que sume K, deberás mostrar el siguiente output:

output	
1	No se encontraron numeros que sumen {K}.

Ejecución

Tu programa se debe compilar con el comando `make` (puedes limpiar el output de compilación con `make clean`) y debe generar un ejecutable de nombre `set-2` que se ejecuta con el siguiente comando:

```
./set-2 INPUT_PATH OUTPUT_PATH
```

donde `INPUT_PATH` será la ruta a un archivo con los eventos a simular y `OUTPUT_PATH` la ruta del archivo donde se guardarán los resultados. Por ejemplo:

```
./set-2 tests/input/test_01.txt outputs/test_01.txt
```

Si quieres verificar que el output generado por tu código es igual al esperado en un test (es decir, tu código pasa el test), puedes hacerlo con el siguiente comando:

```
diff EXPECTED_OUTPUT_PATH PROGRAM_OUTPUT_PATH
```

donde `EXPECTED_OUTPUT_PATH` será la ruta a un archivo con el output esperado del test, y `PROGRAM_OUTPUT_PATH` la ruta del archivo con el output de tu programa para dicho test. Si este comando finaliza sin imprimir nada en consola, significa que los archivos que entregaste son idénticos, y por tanto que el test pasa correctamente. En caso contrario, lo que se imprime es la diferencia entre los dos archivos, lo que podría ayudarte a ver porqué el test falló.

En caso de querer correr el programa para ver los leaks y errores de memoria utilizando `valgrind`, deberás utilizar el siguiente comando:

```
valgrind ./set-2 INPUT_PATH OUTPUT_PATH
```

Por último, en caso de querer correr todos los tests:

```
make test
```

A continuación detallamos los archivos de input y output.

Input

El archivo de input comenzará con el número `E` que corresponde al número de eventos a recibir, y las siguientes líneas corresponden a cada uno de estos eventos. Notar que algunos eventos (como `DICTSORT` y `TWOSUM`) ocupan más de una línea, ya que tras el comando se entregan los `N` elementos correspondientes. Como ejemplo tenemos el siguiente input:

input	
1	6 # Cantidad de eventos
2	DICTSORT 3 4
3	casa
4	arte
5	beso
6	INSERT 5
7	INSERT 8
8	SEARCH 5
9	DELETE 8
10	TWOSUM 4 7
11	3
12	4
13	2
14	10

Output

output

```
1 Se han ordenado las 3 palabras con DictSort:
2 arte
3 beso
4 casa
5 Se ha insertado el numero 5 en el ABB.
6 Se ha insertado el numero 8 en el ABB.
7 Se ha encontrado el numero 5 en el ABB.
8 Se ha eliminado el numero 8 en el ABB.
9 Se encontraron dos numeros que suman 7: 3 y 4.
```